# Autonomous UAV Control in Adversary Scenarios: An Application of Reachability Analysis via Controlled Dynamical Embeddings

António Canteiro

***Abstract*—In a reach-avoid scenario in which a quad-rotor UAV is preventing an attacking UAV from reaching a critical area, the defender must be guaranteed to reach the attacker before the attacker reaches the critical area.**

**Expanding on previous work on defensive control strategies for reach-avoid games, this thesis proposes a novel NMPC based strategy which drives the set of possible defender trajectories to overlap with the set of possible attacker trajectories. Unlike other NMPC and CBF based strategies, the set of possible trajectories is continuously updated according to the quad-rotor dynamics.**

**With the proposed control strategy, the defender continuously maneuvers itself to the trajectory most likely to reach all possible attacker trajectories, thus maintaining a guarantee of area defense throughout its operational period.**

***Index Terms*—quad-rotor, UAV, reach-avoid, control, autonomous, adversarial, NMPC, reachability, embeddings, defense**

## I. Introduction

Reach-Avoid (RA) games are a class of differential games between two adversary agents, wherein an attacker agent aims to reach a target set, while the defender agent aims to intercept the attacker before it reaches its goal. The study of 3-dimensional RA games between quad-rotor UAVs is a pressing issue, since as the presence of UAVs in urban airspaces becomes more prevalent the more disruptions it causes, for example in airport airspaces [1]. In such a scenario, the study of RA games also implies a host of challenges to address, like dealing with non-linear system dynamics, or formulating defense strategies that balance efficacy with computational feasibility.

Since the conception of differential games [2], much work has been done to address the aforementioned challenges, amongst others, in this field of research. Solving the Hamilton-Jacobi-Isaacs (HJI) system of differential equations, for example, is one method to solve RA games that guarantees optimal control, at the cost of being prohibitively computationally expensive for systems of higher dimensions [3] [4] [5]. Constraint Barrier Function (CBF) based strategies condition the input as to ensure the system stays within a set of values which ensure interception under certain conditions [5] [6] [7] [8], but the CBF cannot provide an optimal trajectory, only instantaneous inputs, and it struggles to handle non-linear systems. Considering these limitations, the Non-linear Model Predictive Controller (NMPC), which calculates the state trajectory that minimizes an objective function over a finite time horizon, is well suited for RA and other differential games since it can deal with non-linear dynamics directly and offers great flexibility in choice of objectives and constraints <**empty citation**> [9], however, translating a defense strategy which takes into account the agents' set of possible trajectories into an objective function is challenging.

It is with these advantages and challenges in mind that this work proposes an NMPC-based defensive control strategy for a non-linear RA game. The proposed solution combines a parametric reachable set framework with the KLD in the NMPC objective function. As contributions, this work proposes: 1) an NMPC based defense strategy which provides a best-possible capture solution, presented in III-B; and 2) the safe reachability cone as a tool to validate a defensive trajectory, presented in Section III. The rest of the work is organized as follows: the RA scenario is formulated in Section II, the theorized results are validated in a MATLAB simulated environment in Section IV, and final remarks are made in Section V.

*Notation:* The set of real numbers is denoted by $\mathbb{R}$. The identity matrix is denoted by $\mathbb{I}$. Given a vector $\mathbf{v} \in \mathbb{R}^n$, its transpose is denoted by $\mathbf{v}^T$, and likewise for a matrix $\mathbf{M}$. The inner product between two matrices or vectors is denoted by $\mathbf{v} \cdot \mathbf{M}$. The cross product operation between a vector and a matrix, denoted by $\mathbf{v} \times \mathbf{M}$, is equivalent to the inner product between the skew asymmetric matrix ($SO(3)$) of that vector, which is denoted by $[\mathbf{v}]_\times$, and that matrix. When applied to a matrix, $[\mathbf{M}]_\times$, it can also be used to represent the inverse transformation from an $SO(3)$ matrix to an $\mathbb{R}^3$ vector. The operation vec($\mathbf{M}$) returns the vectorized matrix.

## II. Attacker-Defender Problem Formulation

The conditions of the reach-avoid scenario are firstly formulated, starting with the definition of the quad-rotor model in Subsection II-A. This is followed, in Subsection II-B, by the definition of the attacker's goal and the conditions the defender must satisfy.

*This work was not supported by any organization

[1]Albert Author is with Faculty of Electrical Engineering, Mathematics and Computer Science, University of Twente, 7500 AE Enschede, The Netherlands albert.author@papercept.net

[2]Bernard D. Researcheris with the Department of Electrical Engineering, Wright State University, Dayton, OH 45435, USA b.d.researcher@ieee.org

The scenario in study is a reach-avoid game between a defender drone and an attacker drone. The attacker's goal is to reach a critical area, and it may take any feasible trajectory that accomplishes this goal in reasonable time. The defender's goal is to reach the attacker before it reaches the critical area, despite the exact attacker trajectory not being known ahead of time. The challenge lies in ensuring that the set of possible defender trajectories fully intersects with the set of attacker trajectories at some moment in the future, thus ensuring the defender reaches the attacker and guaranteeing area defense.

### A. Quad-Rotor Model

The quad-rotor drone 6DoF model is built on the assumptions that its mass remains constant, the method of propulsion are four vertical rotors, attitude control is provided by a torque vector and altitude control by a scalar thrust input, it has GNC capability that allows it to discern its position relative to obstacles and the target and path plan, and it is subject to aerodynamic drag. The model is based on the Euler-Angle-Free model described in [10].

The control input to the quad-rotor drone model can be written as $\mathbf{u} \in \mathbb{R}^4$ where $u_1$ is the net body force and $u_2$, $u_3$, $u_4$ are the body moments. The position $\mathbf{p} \in \mathbb{R}^3$ dynamics is defined by the equation

$$\dot{\mathbf{p}} = \mathbf{v}, \tag{1}$$

and the velocity $\mathbf{v} \in \mathbb{R}^3$ dynamics by the equation

$$\dot{\mathbf{v}} = \mathbf{g} + \frac{u_1}{m}\mathbf{A} \cdot \mathbf{e}_z. \tag{2}$$

The rotation matrix $\mathbf{A} \in \mathbb{R}^{3\times3}$ dynamics are expressed by

$$\dot{\mathbf{A}} = -\mathbf{w} \times \mathbf{A}, \tag{3}$$

and the angular velocity $\mathbf{w}^B \in \mathbb{R}^3$ dynamics are expressed as

$$\dot{\mathbf{w}} = -\mathbf{J}^{-1} \cdot (\mathbf{w} \times \mathbf{J} \cdot \mathbf{w}) + \mathbf{J}^{-1} \cdot \begin{bmatrix} u_2 \\ u_3 \\ u_4 \end{bmatrix}, \tag{4}$$

where $\mathbf{J} \in \mathbb{R}^{3\times3}$ is the moment of inertia matrix referenced to the center of mass.

The defender drone model is represented by the state vector $\mathbf{x} = [\mathbf{p}; \mathbf{v}; \mathrm{vec}(\mathbf{A}); \mathbf{w}]$, considered to be a point-mass at the drone's center of mass.

### B. Defender-Attacker Reachability

The defender is a dynamic system $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ with a bounded control input $\mathbf{u}$, and the set that is guaranteed to contain all possible states for $\mathbf{u} \in \mathcal{U}$ at a future time $k$ is called the defender reachable set and is denoted by $\Phi_k$. An accurate but overly conservative $\Phi_k$ for example, is $\Phi_k = \mathbb{R}^n$. Ideally, $\Phi_k$ is as accurate to the system's dynamics as possible, while remaining computationally efficient. The set containing all feasible trajectories from the current time $k = 0$ to the prediction horizon $k = N$ is called the Defender Reachability Cone (DRC), denoted by $\mathcal{D}$, and is considered to be the set of consecutive reachable sets in that time frame, that is, $\mathcal{D} = \{\Phi_k\}, \quad k = [0 : N]$. The Attacker Reachability Cone (ARC) and reachable sets are denoted as $\mathcal{A} = \{\Psi_k\}, \quad k = [0 : N]$. The defender can reach the attacker at any point in the future if the following holds:

$$\exists \Phi_i \in \mathcal{D} : \Phi_i \cap \Psi_i \supseteq \Psi_i, \Psi_i \in \mathcal{A}, 0 \leq i \leq N. \tag{5}$$

Intuitively, this means that if the intersection between the defender and attacker reachable sets ($\Phi_i \cap \Psi_i$) is a superset of the attacker reachable set ($\supseteq \Psi_i$) at some moment in the future ($i$), then the defender can reach all positions the attacker can reach, as long as $\Phi$ is an inner approximation and $\Psi$ is an over-approximation.

### C. Parametric Reachable Sets

Traditionally, reachable sets for linear dynamic systems are approximated using CCGs [11] [12]. However, CCGs tend to be computationally expensive and using them with non-linear systems requires linearizations of the system dynamics. Therefore the parametric reachable set via controlled dynamical embeddings is better suited for this work, as it doesn't require linearizing the model.

The parametric reachable set, referred to as a parametope [13], is defined as

$$((\mathring{\mathbf{x}}, \alpha, y))_\varepsilon := \{\mathbf{x} \in \mathbb{R}^n : \varepsilon(\mathring{\mathbf{x}}, (\alpha^T \cdot \alpha)^{-1}) \leq y\}, \tag{6}$$

where $\mathring{\mathbf{x}}$ is the center, $\alpha$ the parameter matrix and $y$ is the squared radius, and $\varepsilon(\mathbf{q}, \mathbf{Q})$ the function for an ellipsoid with center $\mathbf{q}$ and shape matrix $\mathbf{Q}$, given by

$$\varepsilon(\mathbf{q}, \mathbf{Q}) = (\mathbf{x} - \mathbf{q})^T \mathbf{Q}^{-1} (\mathbf{x} - \mathbf{q}). \tag{7}$$

The parametope is propagated via the controlled dynamic embedding system

$$\dot{\mathring{\mathbf{x}}} = f(\mathring{\mathbf{x}}, \mathring{\mathbf{u}}), \quad \dot{\alpha} = U(\mathring{\mathbf{x}}, \alpha, y), \quad \dot{y} = E(\mathring{\mathbf{x}}, \alpha, y), \tag{8}$$

where the parameter dynamics function is defined as

$$U = -\alpha \mathrm{Df}(\mathring{\mathbf{x}}), \tag{9}$$

where Df is the jacobian, and the radius dynamics function is defined as

$$E = c \cdot y, \tag{10}$$

$c$ being an expansion coefficient. In [13], $c$ is computed using the Interval Mixed Jacobian (IMJ) of the system which, in turn, is given by the natural inclusion function matrix of the system's jacobian, obtained through Interval Analysis (IA) [14]. Tools such as [15] can efficiently build the smallest IMJ [16], however implementation of these tools lays outside the scope of this work.

It is noted that in systems with partly decoupled dynamics, like the quad-rotor UAV which has decoupled altitude and attitude dynamics, the dynamic components most sensitive to disturbances will define the bounds of the IMJ, and the actual bounds of the less sensitive components are lost. This is noted with the quad-rotor UAV embeddings system, where the relevant spatial dynamics are overshadowed by the attitude dynamics.

The following scaling method is therefore proposed to compute $c$, using only the positional dynamics of the drone, and it is given by

$$c(k) = \left( \frac{||\mathbf{p}_{\max}(k) - \mathbf{p}(k)||}{\sqrt{y(k-1)}} - 1 \right) / T_s, \tag{11}$$

where which $\mathbf{p}$ is the translational component of the drone's state, $\mathbf{p}_{\max}$ the furthest possible trajectory in the next time-step, and $T_s$ the sampling period of the simulation.

## III. Defense Controller Formulation

With the defense conditions defined in (5), and the DRC and ARC defined as sets of dynamically evolving parametopes, the proposed defense control solution can be devised.

Firstly, some notions of ellipsoidal calculus and similarity must be introduced. Consider that the spatial ellipsoid of $\Phi$ is denoted by $\Phi^3$ and it is given by the intersection between $\Phi$ and the $\mathbb{R}^3$ hyperplane [17], this is

$$\Phi^3 = \Phi \cap \mathbb{R}^3, \tag{12}$$

and the same applies for $\Psi^3$. Computing the intersection of ellipsoids invariably requires the solution of an optimization problem [18] [19] for each prediction step, at each simulation step, which is computationally untenable.

In lieu of ellipsoid intersection, the KLD is used to quantify ellipsoid similarity. The KLD is traditionally used as a measure of similarity between two gaussian distributions, each described by a positive-definite symmetric covariance matrices, and it is a smooth, always positive, scalar-valued operation that tends to zero when the distributions are identical [20]. Furthermore, the KLD is asymmetric [21]. These characteristics make it a good candidate to quantify similarity between ellipsoids. The KLD from $\Omega_1$ to $\Omega_2$ with centers $\mathbf{q}_1$, $\mathbf{q}_2$ and shape matrices $\mathbf{Q}_1$, $\mathbf{Q}_2$ respectively, and dimensions $n \times n$, has the function notation $\kappa(.,.)$ and is given by

$$\begin{aligned} \kappa(\Omega_1, \Omega_2) = & \frac{1}{2}(\mathtt{tr}(\mathbf{Q}_2^{-1}\mathbf{Q}_2) + (\mathbf{q}_2 - \mathbf{q}_1)^T \mathbf{Q}_2^{-1}(\mathbf{q}_2 - \mathbf{q}_1) \\ & - n + \ln\left(\frac{\det \mathbf{Q}_2}{\det \mathbf{Q}_1}\right)). \end{aligned} \tag{13}$$

The array of pairwise KLD operations between elements of $\mathcal{A}$ and $\mathcal{D}$ is denoted by $K(\mathcal{A}, \mathcal{D})$ such that

$$K(\mathcal{A}, \mathcal{D}) := [\kappa(\Psi_k, \Phi_k)], \quad k = [0:N]. \tag{14}$$

The `min` operator is not continuous, which makes it unsuitable for optimization problems [22]. The LSE function, denoted by $l(.)$, poses a smooth under-approximation of the `min` operator as seen in [23].

Before moving on, a disclaimer is called for. Recalling (5), the defender can reach all positions the attacker can reach at some moment $i$, if $\Phi_i \cap \Psi_i \supseteq \Psi_i$ and $\Phi_i$ is an inner approximation. Since the defender set is defined as a parametope, which is an over-approximation, only the first part of the condition may hold, which does not guarantee interception. However, it is considered that the larger the volume of $\Phi_i^3 \cap \Psi_i^3$, the more likely the defender is to intercept the attacker. With this approach in mind, the proposed control architecture is presented in the following section.

### A. Control Architecture

The controller receives as input the current defender state $\mathbf{x}$ and current DRC $\mathcal{D}$, as well as the current attacker state $\mathbf{z}$, reachable set parameter matrix $\gamma$ and reachable set radius $r$. Before being passed to the solver, this data is transformed as follows.

Knowing the attacker model, $\mathcal{A}^3$ can be pre-computed in full before being passed as parameters to the numerical solver, to ease the computational burden. Computing the full $\mathcal{D}$ trajectory online is impractical however, as the objective function becomes far too complex for the solver to handle, being composed of dozens of thousands of sub-functions. For this reason, an approximated ellipsoid $\Phi'$ is defined in the defender's case, given by

$$\Phi' = \{\mathbf{x} \in \mathbb{R}^3 : \varepsilon(\mathbf{p}, \mathbf{W}) \leq 1\}, \tag{15}$$

$$\mathbf{W} = w(\mathbf{A}, y), \tag{16}$$

where $\mathbf{p}$ is the positional component of the defender model, $\mathbf{A}$ is the rotation matrix component of the defender model, $y$ the squared radius computed with the proposed scaling method and $\mathbf{W}$ the shape matrix given by the function

$$w(\mathbf{A}, y) = \mathbf{A} \cdot \left( y \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0.1 \end{bmatrix} \right) \cdot \mathbf{A}^T. \tag{17}$$

The intuition here is that $\mathbf{W}$ is a disk, aligned according to the defender's dynamics, scaled with the most pessimistic radius given by $y$. The set of all 3-dimensional reduced defender ellipsoids for predicted time-steps is denoted $\mathcal{D}'$, that is $\mathcal{D}' = \{\Phi'_k\}, \quad k = [0:N]$. The data flow diagram for the defender controller is shown in Figure 1.

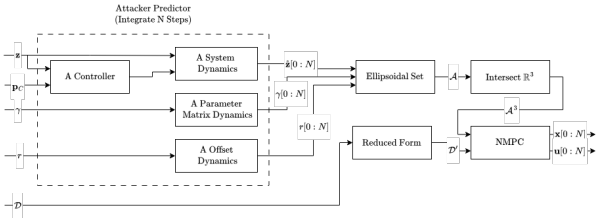



Fig. 1: Data flow diagram of defense-centric controller.

Next in series, the NMPC receives as inputs the $\mathcal{A}^3$ defined previously, and in following, the components of the NMPC objective function components are defined and briefly explained, before joining them together. Consider the minimum KLD from $\Psi_k^3$ to $\Phi'_k$, $k = [0:N]$, to be

$$d_{AD} = l(K(\mathcal{A}^3, \mathcal{D}')), \tag{18}$$

and the symmetric counterpart,

$$d_{DA} = l(K(\mathcal{D}', \mathcal{A}^3)), \tag{19}$$

and assume that both values correspond to the pair of concurrent ellipsoids $\Phi'_i$ and $\Psi_i^3$ closest to each other. In the final

stage of approach, when there is overlap between $\mathcal{D}'$ and $\mathcal{A}^3$, in order for the first part of the reach condition defined in (5) to be satisfied, then the inequality $d_{AD} \leq d_{DA}$ must be verified. Thus, the system should evolve such that $d_{AD}$ decreases faster than $d_{DA}$, which is enforced by minimizing $d_{AD}$ and at the same time maximizing $d_{DA}$ in the objective function.

The relevant minimum KLD is the one between the pair of concurrent ellipsoids which are nearest. This is enforced by introducing the term $e$ in the objective function, which is the minimum of the set of ellipsoid center error norms, denoted as

$$e = l([||\mathbf{p}_k - \mathbf{d}_k||_2]), \quad k = [0:N], \tag{20}$$

where $\mathbf{d}_k$ is the attacker positional variables.

This all comes together in the optimization problem

$$\begin{aligned} \min_{X,U} \quad & eQ_e + d_{AD}Q_{AD} - d_{DA}Q_{DA} + \sum_{k=1}^{N} ||\mathbf{u}_k||_2^2 R \\ s.t.: \quad & \mathbf{x}_0 = \mathbf{x0} \\ & \mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), \\ & \mathbf{W}_k = w(\mathbf{A}_k, y_k), \\ & \mathbf{x} \in \mathcal{X} \\ & \mathbf{u} \in \mathcal{U} \end{aligned} \tag{21}$$

where $N$ is the prediction horizon, $f(.,.)$ is the quad-rotor's dynamics function, and $Q_e$, $Q_{AD}$, $Q_{DA}$ and R are the weight matrices. The optimization variables are $X = [\mathbf{x}_k]$ and $U = [\mathbf{u}_k]$, and the parameters are $\mathbf{x0}$, $\mathbf{y} = [y_k]$ and $\mathcal{A}^3$.

### B. Safe Reachability Cone

The Safe Reachability Cone (SRC) is the set of trajectories predicted to be denied from the attacker, and is a useful tool to validate and visualize a defensive trajectory. It is represented as

$$\mathcal{S} = [\Upsilon_k], \quad i \leq k \leq N, \tag{22}$$

where $\Upsilon_k$ is the safe reachability set at time-step $k$, and $i$ is the time-step the defender is predicted to reach the attacker. $\Upsilon$ is a parametope defined as

$$\Upsilon = ((\mathring{\mathbf{x}}_S, \gamma, r))_\varepsilon, \tag{23}$$

propagated via the controlled dynamic embedding system

$$\dot{\mathring{\mathbf{x}}}_S = q(\mathring{\mathbf{z}}) + \mathbf{x}_o, \quad \dot{\gamma} = U(\mathring{\mathbf{z}}, \gamma, r), \quad \dot{r} = E(\mathring{\mathbf{z}}, \gamma, r), \tag{24}$$

where $q(.)$ is the attacker dynamics, $\mathbf{z}$ the attacker state vector, $\gamma$ the attacker parameter matrix with initial value set at $\alpha_i$, and $r$ is the attacker squared radius with initial value $y_i$. $\mathbf{x}_o$ is an offset which aligns the central predicted attacker trajectory with $\Phi_i$. The SRC in a simplified reach-avoid scenario is illustrated in Figure 2.

## IV. Simulations and Discussion

The developed systems were validated through simulation, implemented in MATLAB [24], using the Runge-Kutta-4 (RK4) [25] integration method and a time-step length of $0.01s$. The simulated quad-rotor drones have the physical parameters defined in Table I and the input bounds defined in Table II.

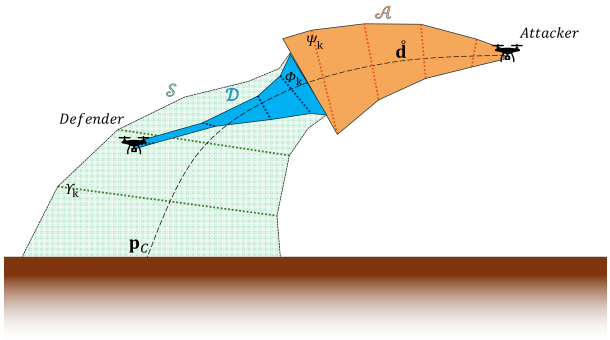



Fig. 2: Illustration of safeguarded area: 2-dimensional simplification. For $k = [0:N]$, $RD = [\Phi_k]$ is the DRC, $\mathcal{A} = [\Psi_k]$ is the ARC, and $\mathcal{S} = [\Upsilon_k]$ is the SRC.

### A. Reachable Set Validation

A simulation of a quad-rotor model moving from an initial hovering position to another hovering position was run with MATLAB [24] to verify the accuracy of the reachable set, outlined in Section II-C, when compared to potential trajectories. Predicted state trajectories are computed from the current moment to the prediction horizon by fixing the drone's torque input $[\mathbf{u}_2; \mathbf{u}_3; \mathbf{u}_4]$ as combinations of the maximum, minimum, and middle values, and the $\mathbf{u}_1$ input at its minimum or maximum.

In both the following figures, the $k_{sim}$ axis indicates the real-time time-steps, and the $k_{pred}$ axis indicates the predicted time-steps. Figure 3 shows the median distance between the reachable set surface and the trajectories it contains [26] at the respective time-steps, where positive values indicate the trajectory is contained in the set, and negative values that it's outside. Figure 4 shows the volume difference between the reachable set and the convex hull formed by the trajectories.

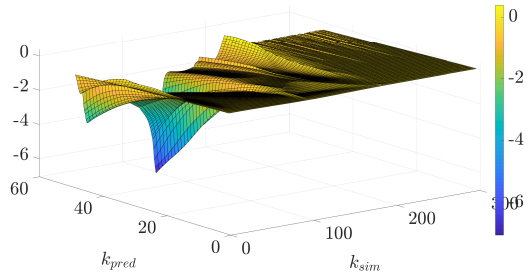



Fig. 3: Median distance between reachable set surface and potential predicted trajectories.

### B. Two Drone Scenario

A reach-avoid scenario is simulated with MATLAB [24], in which the attacker drone moves toward the critical area, set at the origin, and has a polynomial trajectory planner in series with a Mellinger controller [27] as the control architecture. Furthermore, the attacker's trajectory is made unpredictable

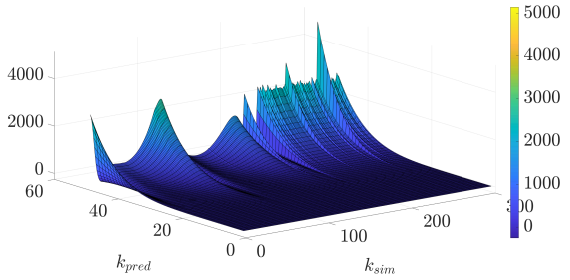



Fig. 4: Volume difference between reachable set and convex hull of trajectories.

TABLE I: Quad-Rotor Parameters

| Parameter | Description | Units | Value |
|---|---|---|---|
| $M$ | Total Mass | kg | 0.5 |
| $L$ | Arm Length | m | 0.25 |
| $I_x$ | Angular Momentum x axis | kg·m$^2$/s | 0.0196 |
| $I_y$ | Angular Momentum y axis | kg·m$^2$/s | 0.0196 |
| $I_z$ | Angular Momentum z axis | kg·m$^2$/s | 0.0264 |

TABLE II: Quad-Rotor Input Bounds

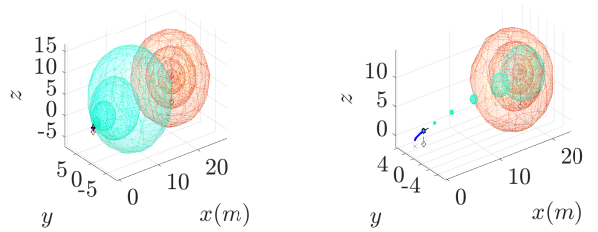
| Variable | Units | Lower Bound | Upper Bound |
|---|---|---|---|
| $u_1$ | N | 0.0001 | 50 |
| $u_2$ | rad·s$^{-2}$ | $-2\pi$ | $2\pi$ |
| $u_3$ | rad·s$^{-2}$ | $-2\pi$ | $2\pi$ |
| $u_4$ | rad·s$^{-2}$ | $-2\pi$ | $2\pi$ |

by repeatedly shifting the target waypoint by random amounts. The defender drone is initially hovering above the critical area.

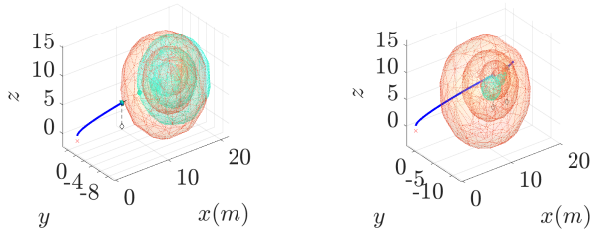
A series of images showing this simulation from the initial moment to the moment the defender reaches the attacker are in Figure 5, where $\mathcal{D}^3$ are the blue ellipsoids and $\mathcal{A}^3$ are the orange ellipsoids. The simulation shows the defender speeding towards the attacker, then as $\mathcal{D}^3$ and $\mathcal{A}^3$ overlap the defender slows down as there is a $\Phi_i^3$ which expands to contain the corresponding $\Psi_i^3$. This can be seen as the defender positioning itself such that there is a moment it can reach any trajectory the attacker may take.

The defender's performance data is shown in Figure 6, where the top graph shows the norm of the distance between defender and attacker throughout the simulation, and the bottom graph shows the area of $\mathcal{A}^3 \cap \mathcal{G}$ in blue, and $\mathcal{S}^3 \cap \mathcal{G}$ in orange, denoted by $\Theta_A$ and $\Theta_S$ respectively, with $\mathcal{G}$ being the $\mathbb{R}^3$ ground plane. It can be seen that $\Theta_S$ fully overlaps $\Theta_A$ from approximately $t = 0.1s$ to $t = 0.3s$, which indicates that throughout this timeframe the defender is positioned such that it can reach all attacker trajectories.

The evolution of the KLD between the 3-dimensional ARC and DRC is shown in Figure 7 as a surface, where the columns are the $K(\mathcal{A}^3, \mathcal{D}')$ at each simulation time-step, the values being indicated by the color bar, capped at 1000. The red line highlights the minimum KLD at each $k_{pred}$. It can be seen that the KLD tends to decrease to zero, indicating that the DRC is overlapping with the ARC. Observing red line, it can be seen that the $k_{pred}$ of the minimum KLD tends to decrease, showing how the predicted time of interception approaches the present.




(a) $t = 0.01s$ (b) $t = 0.27s$




(c) $t = 0.54s$ (d) $t = 0.81s$

Fig. 5: Visualization of single defender with defense-centric controller vs unpredictable attacker with Mellinger controller in defense scenario, N=50.

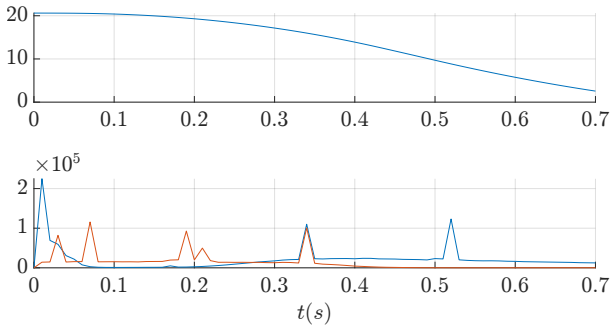



Fig. 6: Performance analysis of defense-centric controller in defense scenario with 1 defender and 1 unpredictable attacker.

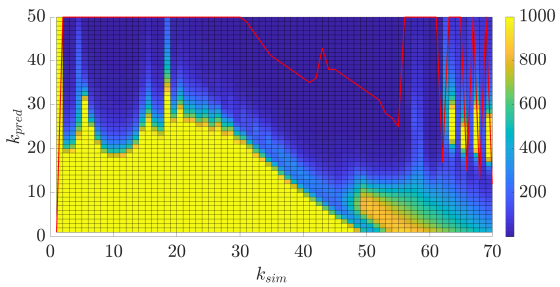



Fig. 7: KLD between A and D across simulation.

## V. Conclusions and Further Work

This work tackles the issue of non-linear quad-rotor UAV control in a reach-avoid game. The simulation results in Section IV show that the proposed control strategy correlates with the defender continuously positioning itself such that there is a moment it can reach any trajectory the attacker may take.

In future works, the parametric reachable set propagation could be refined, by using more sophisticated interval arithmetic tools for example, to improve the accuracy of the reachable sets. Furthermore, physical experiments could be conducted to validate the proposed controller in the real world. A tangential, but perhaps more interesting line of research would be to apply the reachability-based control philosophy here proposed to more constructive scenarios, such as search-and-rescue out at sea.

## References


[1] P. Tamsin, "Fact-checking europe's 'drone problem': Why are airports closing over drone sightings?" *Euronews*, 2025-11-20. [Online]. Available: https://www.euronews.com/2025/11/20/fact-checking-europes-drone-problem-why-are-airports-shuttering-over-drone-sightings.

[2] R. Isaacs, *Differential games: a mathematical theory with applications to warfare and pursuit, control and optimization*. Courier Corporation, 1999.

[3] Z. Zhou, W. Zhang, J. Ding, H. Huang, D. M. Stipanović, and C. J. Tomlin, "Cooperative pursuit with voronoi partitions," *Automatica*, vol. 72, pp. 64–72, 2016.

[4] S. Y. Hayoun and T. Shima, "On guaranteeing point capture in linear n-on-1 endgame interception engagements with bounded controls," *Automatica*, vol. 85, pp. 122–128, 2017.

[5] M. Bui, S. P. Monckton, and M. Chen, "Reach-avoid differential game with reachability analysis for uavs: A decomposition approach," *ArXiv*, vol. abs/2512.22793, 2025. [Online]. Available: https://api.semanticscholar.org/CorpusID:284311075.

[6] A. M. Ali, H. A. Hashim, and C. Shen, "Mpc based linear equivalence with control barrier functions for vtol-uavs," in *2024 American Control Conference (ACC)*, 2024, pp. 1–6. DOI: 10.23919/ACC60939.2024.10644310.

[7] A. Davydov, P. Rivera-Ortiz, and Y. Diaz-Mercado, "Pursuer coordination in multi-player reach-avoid games through control barrier functions," in *2021 American Control Conference (ACC)*, IEEE, 2021, pp. 3222–3227.

[8] L. Chen and Y. Xia, "Cooperative capture strategy for a group of slower uavs to intercept one faster intruder in 3-d space," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 61, no. 4, pp. 8757–8769, 2025.

[9] D. Novak and S. Tebbani, "A comparative study of nmpc strategies for prioritized multi-uav trajectory tracking with collision avoidance in agricultural field mapping missions," pp. 1195–1200, 2024. DOI: 10.1109/CoDIT62066.2024.10708571.

[10] A. El Asslouj and H. Rastgoftar, "Quadcopter tracking using euler-angle-free flatness-based control," pp. 1–6, 2023.

[11] F. Rego and D. Silvestre, "Explicit computation of guaranteed state estimates using constrained convex generators," pp. 1400–1405, 2024-12. DOI: 10.1109/CDC56724.2024.10885976.

[12] D. Silvestre, "Comparison of recent advances in set-membership techniques: Application to state estimation, fault detection and collision avoidance," *European Journal of Control*, vol. 81, p. 101 161, 2025, ISSN: 0947-3580. DOI: https://doi.org/10.1016/j.ejcon.2024.101161. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0947358024002218.

[13] A. Harapanahalli and S. Coogan, "Parametric reachable sets via controlled dynamical embeddings," pp. 2694–2700, 2025.

[14] L. Jaulin, M. Kieffer, O. Didrit, and E. Walter, "Interval analysis," pp. 11–43, 2001. DOI: 10.1007/978-1-4471-0249-6_2. [Online]. Available: https://doi.org/10.1007/978-1-4471-0249-6_2.

[15] A. Harapanahalli, S. Jafarpour, and S. Coogan, "Immrax: A parallelizable and differentiable toolbox for interval analysis and mixed monotone reachability in jax," *IFAC-PapersOnLine*, vol. 58, no. 11, pp. 75–80, 2024, 8th IFAC Conference on Analysis and Design of Hybrid Systems ADHS 2024, ISSN: 2405-8963. DOI: https://doi.org/10.1016/j.ifacol.2024.07.428. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2405896324005275.

[16] A. Harapanahalli and S. Coogan, "A linear differential inclusion for contraction analysis to known trajectories," *IEEE Transactions on Automatic Control*, vol. 71, no. 3, pp. 2077–2084, 2026. DOI: 10.1109/TAC.2025.3626266.

[17] A. A. Kurzhanskiy and P. Varaiya, "Ellipsoidal toolbox (et)," pp. 1498–1503, 2006. DOI: 10.1109/CDC.2006.377036.

[18] S. Yousefi, X.-W. Chang, H. Wymeersch, B. Champagne, and G. Toussaint, "A novel approach for ellipsoidal outer-approximation of the intersection region of ellipses in the plane," *Computational Optimization and Applications*, vol. 69, no. 2, pp. 383–402, 2018.

[19] L. Ros, A. Sabater, and F. Thomas, "An ellipsoidal calculus based on propagation and fusion," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 32, no. 4, pp. 430–442, 2002.

[20] J. Duchi, "Derivations for linear algebra and optimization," *Berkeley, California*, vol. 3, no. 1, pp. 2325–5870, 2007.

[21] A. Łysiak and M. Szmajda, "Empirical comparison of the feature evaluation methods based on statistical measures," *IEEE Access*, vol. 9, pp. 27 868–27 883, 2021-02. DOI: 10.1109/ACCESS.2021.3058428.

[22] Y. Gilpin, V. Kurtz, and H. Lin, "A smooth robustness measure of signal temporal logic for symbolic control," *IEEE Control Systems Letters*, vol. 5, no. 1, pp. 241–246, 2021. DOI: 10.1109/LCSYS.2020.3001875.

[23] L. Lindemann and D. V. Dimarogonas, "Control barrier functions for signal temporal logic tasks," *IEEE Control Systems Letters*, vol. 3, no. 1, pp. 96–101, 2019. DOI: 10.1109/LCSYS.2018.2853182.

[24] T. M. Inc., *Matlab version: 9.13.0 (r2022b)*, Natick, Massachusetts, United States, 2022. [Online]. Available: https://www.mathworks.com.

[25] W. H. Press, B. Flannery, S. Teukolsky, and W. Vetterling, "Section 17.1 runge-kutta method," *Numerical Recipes: The Art of Scientific Computing*, 2007.

[26] D. Eberly, "Distance from a point to an ellipse, an ellipsoid, or a hyperellipsoid," Ph.D. dissertation, Geometric Tools, 2020.

[27] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," pp. 2520–2525, 2011. DOI: 10.1109/ICRA.2011.5980409.